

Week 08

Dynamic Data & Fetch API

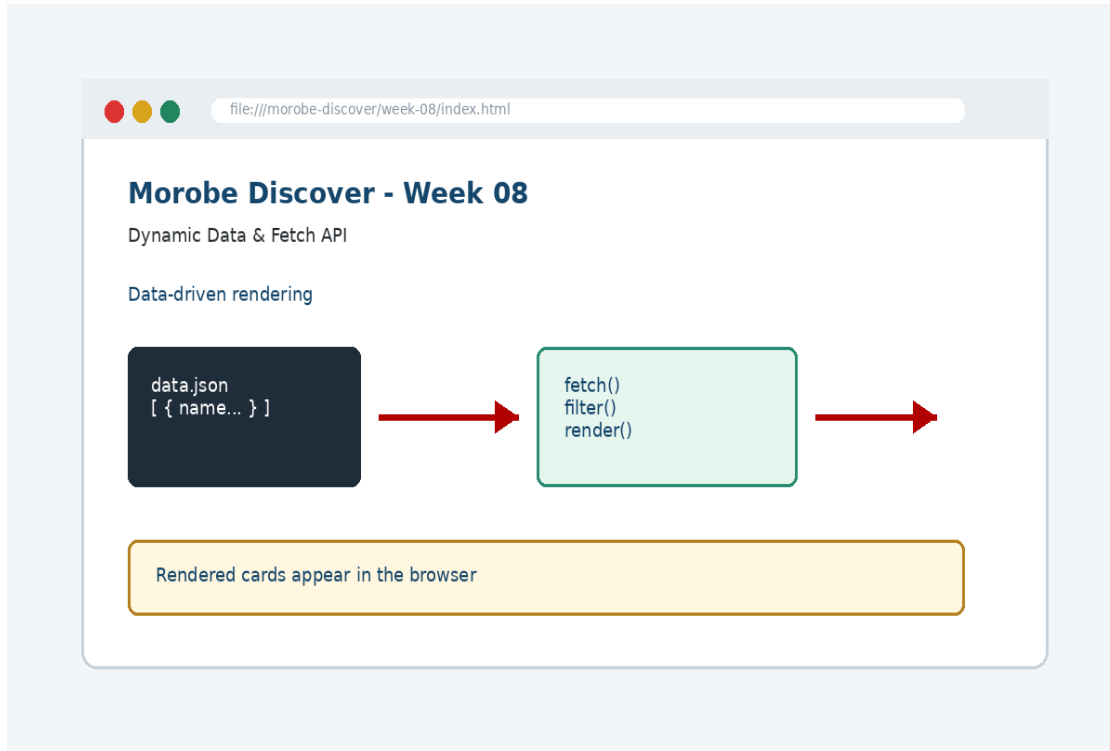
Decouple content from structure by loading and rendering dynamic external data.



Learning outcomes

Decouple content from structure by loading and rendering dynamic external data.
Explain the core programming/design concept for this week
Read and modify key code patterns
Test the weekly project increment
Prepare evidence for GitHub submission

Today's problem



Why it matters

Store repeatable content as data, fetch it, then render the interface from that data.

Project before -> after

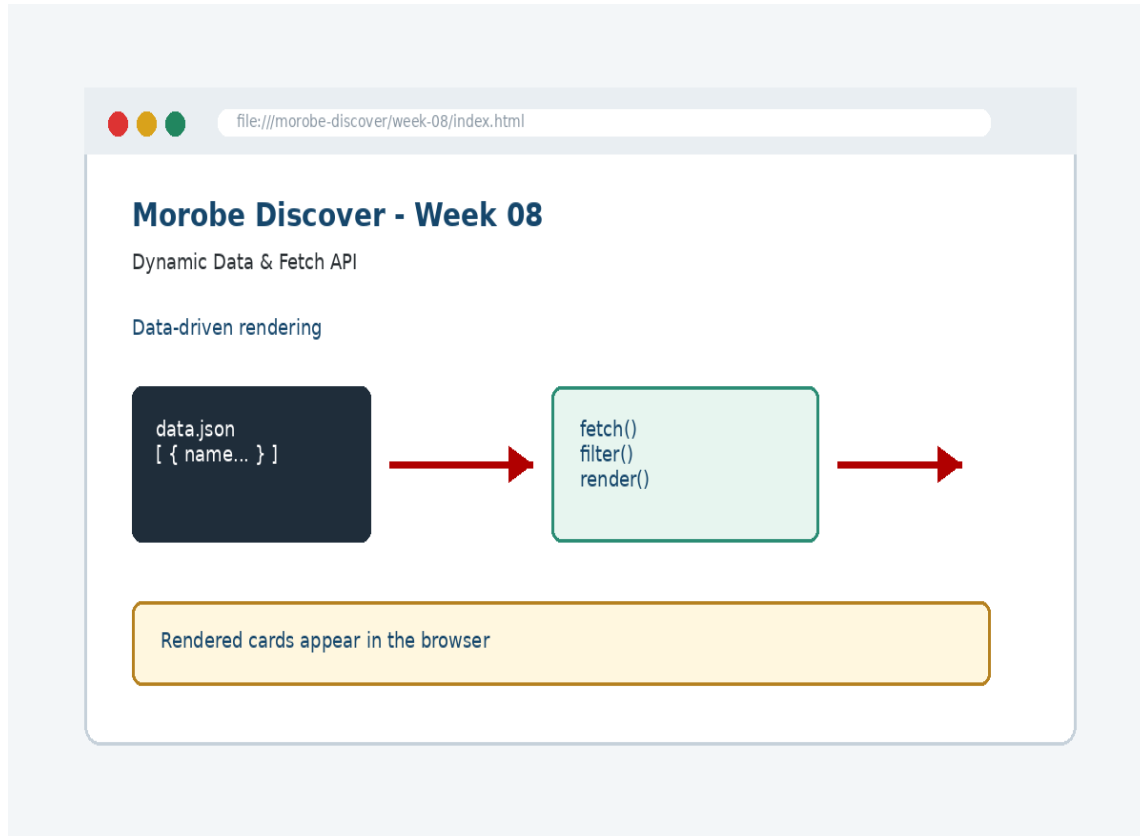
Before Week 08

Week 07 added validation and saved lightweight user choices.

After Week 08

Destination and event cards are generated from JSON data instead of repeated static HTML.

What we build this week



Create destinations.json
Fetch the JSON file from JavaScript
Render cards into the DOM
Add search or category filtering
Show loading and error states

Core principle

Store repeatable content as data, fetch it, then render the interface from that data.

Keep in mind

A good web developer can point to the code, the browser result, and the user reason behind the decision.

Key concepts

Concept 1

JSON represents structured content

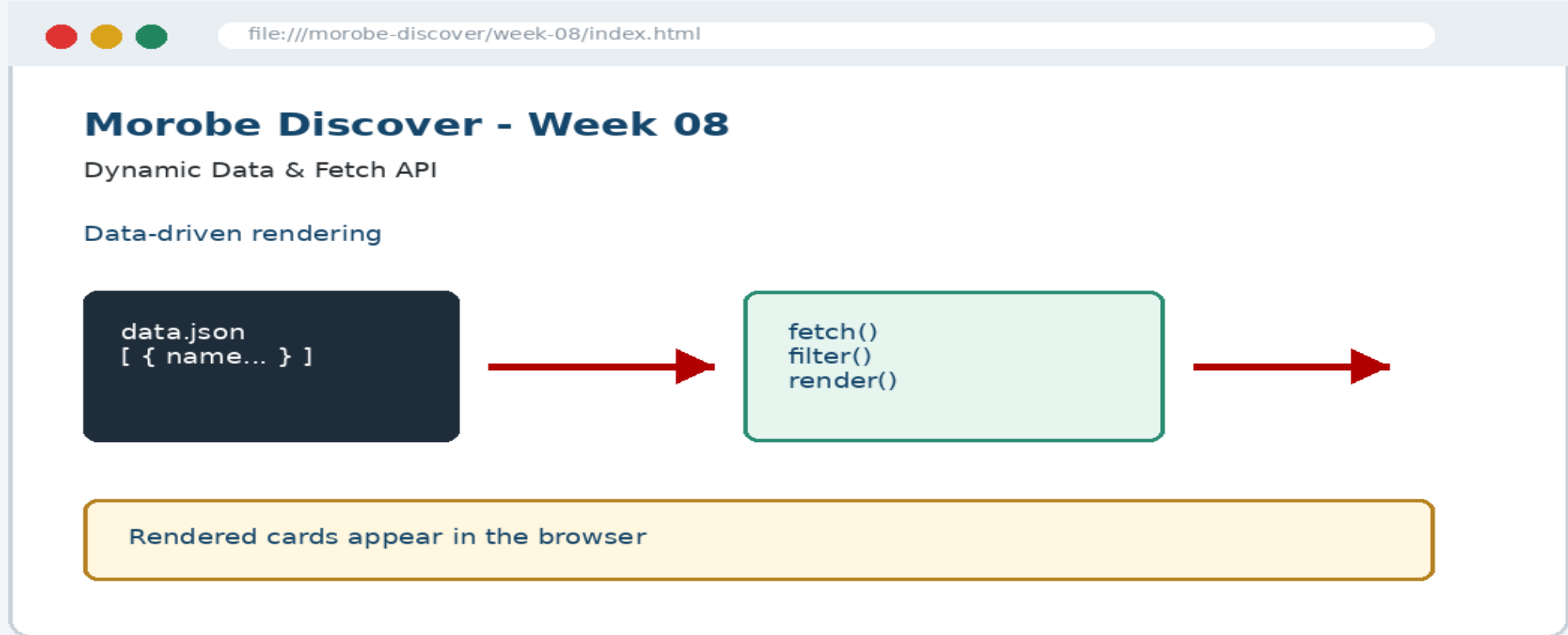
Concept 2

`fetch()` loads external resources asynchronously

Concept 3

Filtering transforms data before rendering

Project anatomy



Key code pattern A

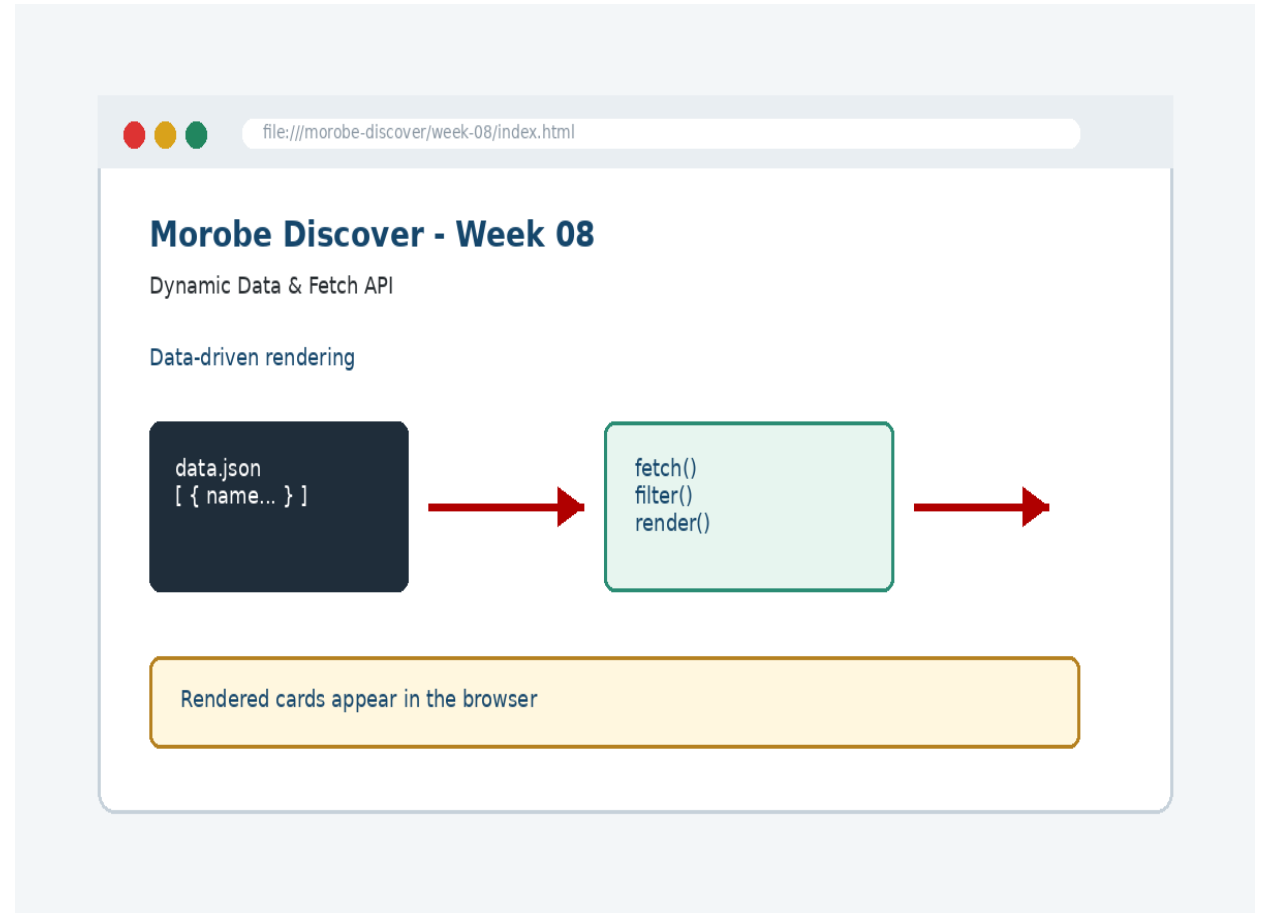
```
const response = await
fetch("data/destinations.json");
if (!response.ok) throw new Error("Data failed
to load");
const destinations = await response.json();
```

Explain it

Which file contains this code?
Which line creates the visible change?
What should appear in the browser?

Code -> visual result

```
const response = await
fetch("data/destinations.json");
if (!response.ok) throw new Error("Data
failed to load");
const destinations = await
response.json();
```



What changed?

Project file changed for Dynamic Data & Fetch API

The browser output is now closer to the required weekly evidence

The change supports a user task or quality requirement

The change can be tested and committed

Key code pattern B

```
const html = destinations
  .map(item => `
```

Why this pattern is needed

The second pattern usually applies the same idea to a different component, state, layout or data flow.

Apply it to Morobe Discover

Create destinations.json
Fetch the JSON file from JavaScript
Render cards into the DOM
Add search or category filtering



Compare the approaches

Weak approach

- Memorise the definition
- Copy code without understanding
- No browser evidence
- No test note

Professional approach

- Explain the concept
- Apply the correct code pattern
- Inspect the result
- Commit and describe the evidence

Common mistake

Mistake

The page appears unchanged, but the student assumes the feature is complete.

Correction

Check the correct file, refresh the browser, inspect DevTools, and verify the expected output.

Why does this fail?

```
// Broken checkpoint  
// File changed, but the browser result does not  
change.  
// Possible causes:  
// - wrong file  
// - file not linked  
// - syntax error  
// - browser not refreshed
```

Class question

Which cause would you check first, and why?

Fix the problem

- Confirm the browser is opening the correct file
- Check links to CSS/JS/data files
- Inspect the console for errors
- Use a small working example
- Test again before committing

Class activity

Activity - 7 minutes

Explain what is gained by moving destination content from HTML into JSON.

Work in pairs
Point to the project evidence
Prepare one answer to share

Implementation steps 1-3

Create destinations.json
Fetch the JSON file from JavaScript
Render cards into the DOM

Implementation steps 4-5

Add search or category filtering
Show loading and error states

LIVE DEMO

Demo checklist

1. Open the current project
2. Locate the affected file
3. Add or modify the key code
4. Refresh the browser
5. Inspect the result
6. Explain the change



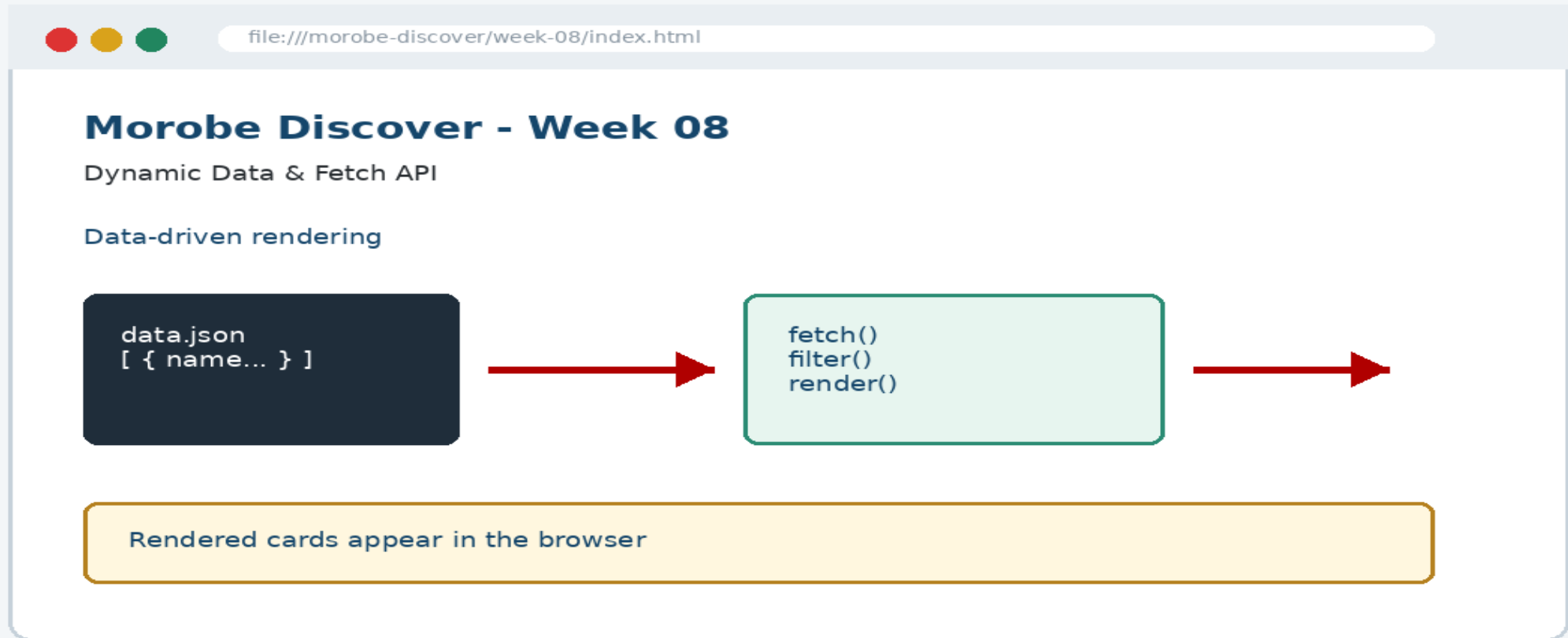
Predict the output

```
const html = destinations
  .map(item => `
```

Prediction prompt

What should change on screen when this code is applied correctly?

Browser result



Testing the feature

- Run through a local web server
- Break the JSON path and observe the error state
- Test filters with empty results
- Check rendered cards preserve accessibility labels

Inspect with browser tools

Inspection path

Right click -> Inspect
Elements panel -> structure/classes
Console -> JavaScript errors
Network/Application -> data/storage when relevant

Evidence to save

Screenshot
Short test note
Changed files
Git commit message

Git evidence

```
git add .  
git commit -m "Week 08 Dynamic Data & Fetch API  
project increment"
```

Why it matters

Git history shows progression from Week 01 to final deployment. It is part of the assessment evidence.

Complete source code

Repository:
`https://github.com/YOUR-USERNAME/is229-morobe-discover`

Weekly folder:
`/week-08/`

Complete project:
`/complete-project/`

Reminder

Do not paste a whole file into a slide. Use the repository for complete code.

Mini challenge

Challenge

Modify one part of the Week 08 implementation and explain what changed in the browser.

State the file edited

State the line/section changed

Show the before and after result

What counts as evidence?

Technical evidence

Changed code files
Browser screenshot
Test note
Git commit

Understanding evidence

Code explanation
Why this method fits
Connection to previous week
Connection to next week

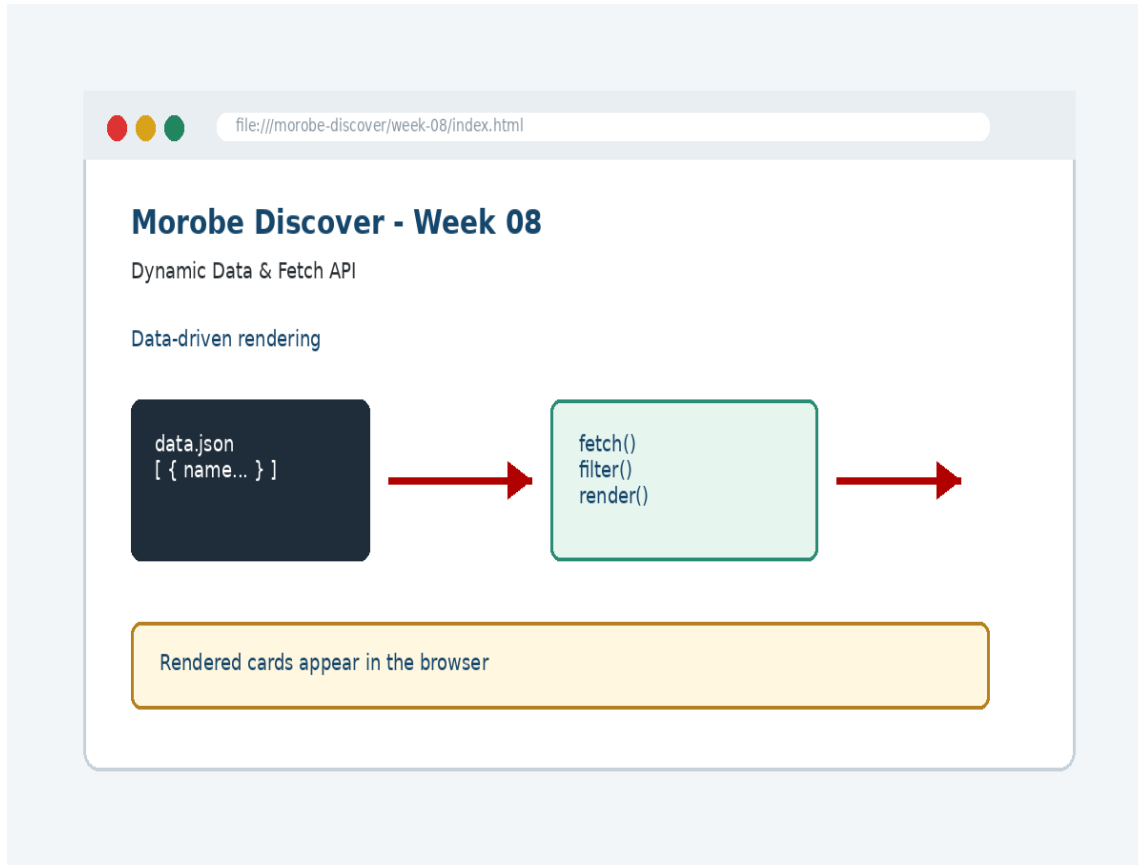
Project lab handoff

- Create destinations.json
- Fetch the JSON file from JavaScript
- Render cards into the DOM
- Add search or category filtering
- Show loading and error states

Definition of done

Week 08 feature is implemented
Project still preserves earlier week features
Browser result matches the expected output
Test note and screenshot are saved
Git commit message is meaningful

What we built today



Destination and event cards are generated from JSON data instead of repeated static HTML.
Evidence: JSON content, Fetch API requests and data filters
Technology: JSON / Fetch API

Lecture summary

Store repeatable content as data, fetch it, then render the interface from that data.

JSON represents structured content

fetch() loads external resources asynchronously

Testing and Git are part of development, not extras

Prepare for next class

Finish this week's lab before the next class
Read the next topic in advance
Bring the current project ready to extend
Be ready to explain one code decision

Exit question

How did Week 08 improve Morobe Discover, and what will the next week build on?